



LAPORAN PRAKTIKUM

Pengembangan Aplikasi Web II

Disusun oleh:

Nama : Adit Yudha Pratama

NIM : 2428240163

Kelas : SI5C

**PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027**

Identitas Praktikum

Nama Mahasiswa	Adit Yudha Pratama
NPM	2428240163
Kelas	SI5C
Pertemuan Ke-	6
Topik / Judul Praktikum	Tugas 1 — RESTful API Murni dengan Express.js (Topik 37: Platform Audio – Podcast)
Tanggal Pelaksanaan	29 September 2026
Dosen Pengampu	Nur Rachmat, M.Kom.

I. Tujuan Praktikum

1. Membangun project Express.js dengan satu file app.js, dependency express dan devDependency nodemon, serta menjalankan server pada port 3000 (atau process.env.PORT) dengan middleware express.json().
2. Menerapkan routing RESTful untuk resource /podcasts dengan 6 endpoint (ambil semua data, ambil satu data, tambah, ubah, hapus, dan filter dengan query string) sesuai konvensi penamaan URL.
3. Menggunakan method HTTP GET, POST, PUT, dan DELETE dengan validasi field wajib serta status code yang tepat (200, 201, 400, dan 404).
4. Menghasilkan response JSON murni tanpa tampilan HTML, dengan format { status, message, data } untuk POST, PUT, DELETE, dan error.
5. Mempublikasikan project ke repository GitHub publik dan men-deploy-nya ke Vercel, lalu menguji seluruh endpoint pada localhost dan URL Vercel.

II. Dasar Teori

REST (*Representational State Transfer*) adalah gaya arsitektur perangkat lunak yang diperkenalkan oleh Fielding (2000) untuk sistem terdistribusi berbasis web. Dalam REST, setiap sumber daya (*resource*) diidentifikasi oleh URL dan dimanipulasi melalui representasinya. Dengan demikian, RESTful API adalah layanan yang mengelola resource melalui URL dan method HTTP, lalu membalas dengan data, bukan halaman web. Pada tugas ini resource yang dikelola adalah podcast dengan endpoint /podcasts.

Setiap method HTTP memiliki semantik yang berbeda (Fielding et al., 2022): GET untuk membaca resource tanpa mengubahnya, POST untuk membuat resource baru, PUT untuk mengganti seluruh representasi resource, dan DELETE untuk menghapus resource. Hasil setiap operasi dinyatakan melalui status code, misalnya 200 (OK), 201 (Created), 400 (Bad Request), dan 404 (Not Found).

Data dipertukarkan dalam format JSON (*JavaScript Object Notation*) yang ringan serta mudah dibaca oleh manusia maupun mesin (Bray, 2017). Seluruh response API pada tugas ini, termasuk response error dan 404, berformat JSON.

Express.js adalah framework web minimalis untuk Node.js yang menyediakan routing (`app.get`, `app.post`, `app.put`, `app.delete`) dan middleware (Express, n.d.). Middleware `express.json()` membaca body request JSON ke `req.body`, `req.params` menyimpan route parameter seperti `:id`, dan `req.query` menyimpan query string yang dipakai untuk filter data. Middleware yang dipasang setelah seluruh route dapat menangani endpoint yang tidak terdaftar dengan response 404.

Git dan GitHub digunakan untuk pengelolaan versi kode, sedangkan Vercel adalah platform yang menjalankan aplikasi Express secara *serverless* (Vercel, n.d.). Karena data pada tugas ini hanya disimpan dalam array di memori, data dapat kembali ke kondisi awal setelah beberapa saat ketika aplikasi berjalan di Vercel.

III. Alat dan Bahan

No.	Nama Alat / Software	Versi / Keterangan
1	Node.js	v24.16.0
2	Visual Studio Code	Version: 1.139.1 (user setup) Commit: 04c0d99f4fb0d8afe6ce4f0c58e31e183ac3e4b1 Date: 2026-09-25T03:40:57Z Electron: 43.6.0 ElectronBuildId: 15274685 Chromium: 150.0.7871.250 Node.js: 24.20.0 V8: 15.0.245.31-electron.0 @github/copilot: 1.0.85.r35379093703.g3514c9a @github/copilot-sdk: 1.0.15.35393089353.gfc44743 OS: Windows_NT x64 10.0.26200— code editor
3	Express.js	^5.2.1 (dependency, sesuai package.json)
4	nodemon	^3.1.14 (devDependency) — restart otomatis saat pengembangan
5	npm	11.13.0
6	Thunder Client	Version 2.41.4— pengujian endpoint
7	Git dan GitHub	— version control dan repository publik
8	Vercel	Platform deployment (serverless)

IV. Langkah Kerja

1. Membuat folder project, lalu menjalankan `npm init -y`, `npm install express`, dan `npm install -D nodemon`. Pada `package.json`, entry point diatur `"main": "app.js"` dengan script `"start": "node app.js"` dan `"dev": "nodemon app.js"`.
2. Membuat file `app.js` yang memuat `express`, menentukan PORT (3000 atau `process.env.PORT`), memasang `express.json()`, membuat array `podcasts` dengan 3 data awal beserta variabel `nextId = 4`, dan membuat route GET `/` yang menampilkan info API (nama, NPM, nomor topik, dan daftar endpoint).
3. Membuat route baca data: GET `/podcasts` (dengan filter `?kategori=` melalui `req.query`) dan GET `/podcasts/:id` (mengembalikan 404 bila id tidak ada, masing-masing diberi komentar method, URL, dan contoh body).
4. Membuat route POST `/podcasts` (validasi field wajib judul, host, dan kategori; id otomatis dengan `nextId++`; status 201), PUT `/podcasts/:id` (404 bila data tidak ada, 400 bila field wajib kosong), dan DELETE `/podcasts/:id` (404 bila data tidak ada).
5. Menambahkan middleware catch-all 404 ("Endpoint tidak ditemukan") dan error handler JSON di akhir file, lalu menjalankan server dengan `npm run dev`.
6. Menguji 10 skenario pada tabel pengujian menggunakan Thunder Client di `http://localhost:3000` dan menyimpan screenshot request, response, serta status code.
7. Membuat `.gitignore` berisi `node_modules/`, menjalankan `git init`, melakukan commit bertahap, dan melakukan push ke repository GitHub publik `tugas1-restful-2428240163`.
8. Menyesuaikan `app.js` agar berjalan di Vercel (`app.listen` hanya dijalankan bila `NODE_ENV` bukan `production` dan aplikasi diekspor dengan `module.exports = app`), mengimpor repository ke Vercel (Add New → Project), lalu men-deploy dan menguji seluruh endpoint pada URL Vercel.

V. Hasil dan Pembahasan

Seluruh pengujian dilakukan pada `http://localhost:3000` dan pada URL Vercel menggunakan Thunder Client Tabel 1 merangkum 10 skenario pengujian yang diminta; kolom status hasil dan keterangan diisi berdasarkan hasil pengujian sebenarnya.

Tabel 1. Hasil pengujian endpoint `/podcasts`

No	Method	Endpoint	Data request	Status diharapkan	Status hasil	Keterangan
1	GET	<code>/podcasts</code>	—	200	200	Berhasil menampilkan seluruh data podcast

No	Method	Endpoint	Data request	Status diharapkan	Status hasil	Keterangan
2	GET	/podcasts/1	—	200	200	Berhasil menampilkan data podcast dengan ID 1
3	GET	/podcasts/99	—	404	404	Berhasil mengembalikan 404 karena data dengan ID 99 tidak ditemukan
4	GET	/podcasts?kategori=pendidikan	—	200	200	Berhasil menampilkan data podcast berdasarkan kategori pendidikan
5	POST	/podcasts	body lengkap	201	201	Berhasil menambahkan data podcast baru
6	POST	/podcasts	field wajib kosong	400	400	Gagal menambahkan data karena field wajib kosong
7	PUT	/podcasts/1	body lengkap	200	200	Berhasil memperbarui data podcast dengan ID 1
8	PUT	/podcasts/99	body lengkap	404	404	Gagal memperbarui data karena podcast dengan ID 99 tidak ditemukan
9	DELETE	/podcasts/1	—	200	200	Berhasil menghapus data podcast dengan ID 1

No	Method	Endpoint	Data request	Status diharapkan	Status hasil	Keterangan
10	DELETE	/podcasts/99	—	404	404	Gagal menghapus data karena podcast dengan ID 99 tidak ditemukan

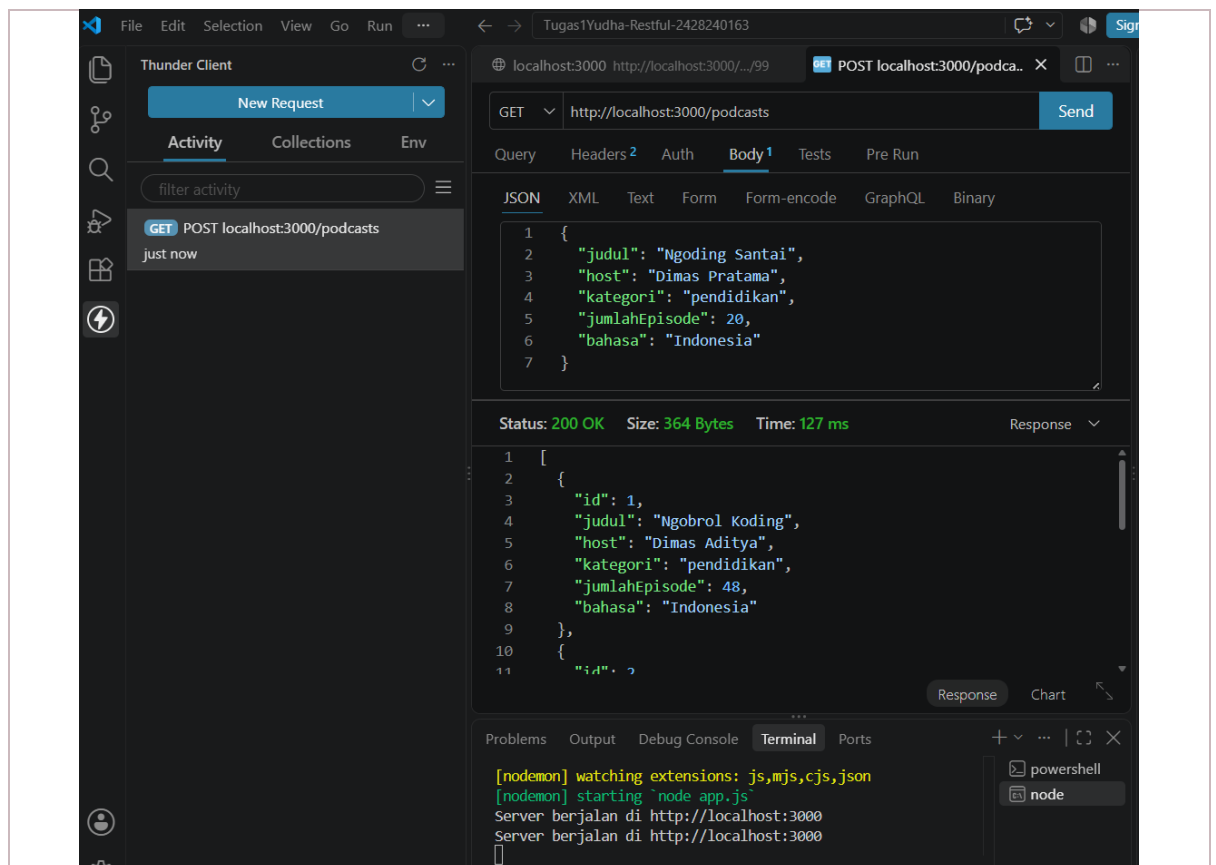
POST /podcasts (body lengkap): { "judul": "Teknologi Santai", "host": "Rina Kusuma", "kategori": "teknologi", "jumlahEpisode": 10, "bahasa": "Indonesia" }

POST /podcasts (field wajib kosong): { "judul": "", "host": "", "kategori": "" }

PUT /podcasts/1 (body lengkap): { "judul": "Ngobrol Koding Season 2", "host": "Dimas Aditya", "kategori": "pendidikan", "jumlahEpisode": 52, "bahasa": "Indonesia" }

Hasil Pengujian dan Pembahasan

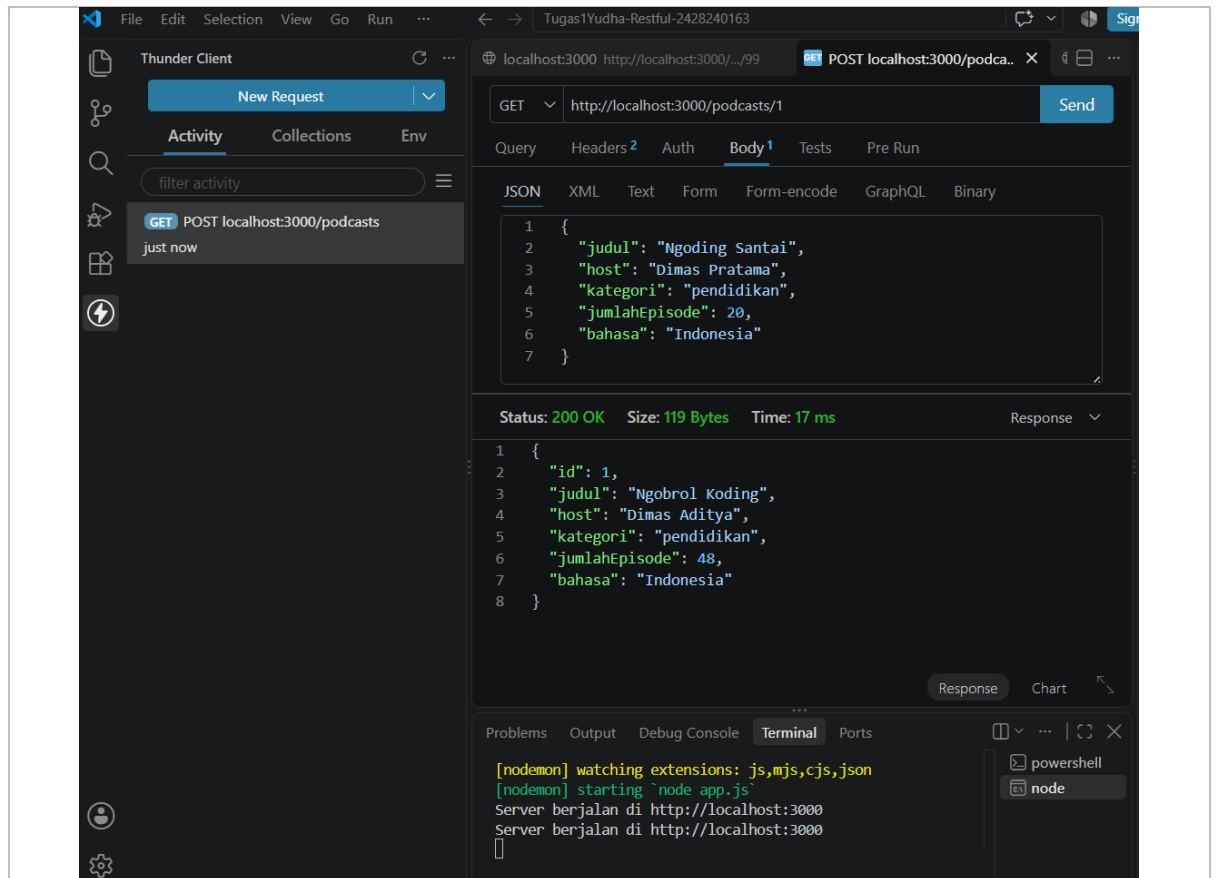
Pengujian 1 — GET /podcasts (ambil semua data)



Gambar 1. GET /podcasts (ambil semua data) (status diharapkan 200)

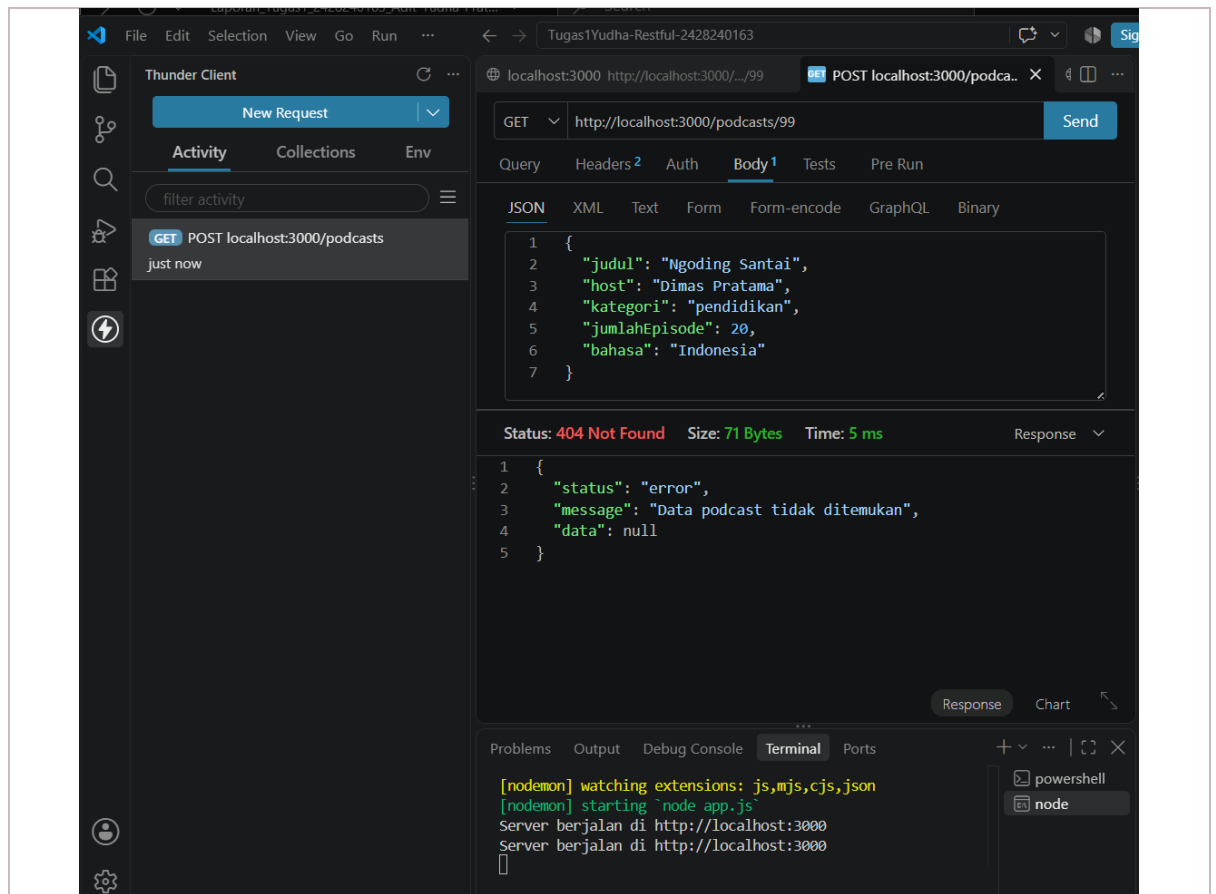
Pembahasan: Method GET dipilih karena operasi ini hanya membaca data tanpa mengubah isi array podcasts. Server membalas status 200 (OK) dengan body berupa array JSON yang berisi seluruh podcast, langsung tanpa pembungkus status dan message, sesuai format response GET pada ketentuan tugas.

Pengujian 2 — GET /podcasts/1 (ambil satu data)



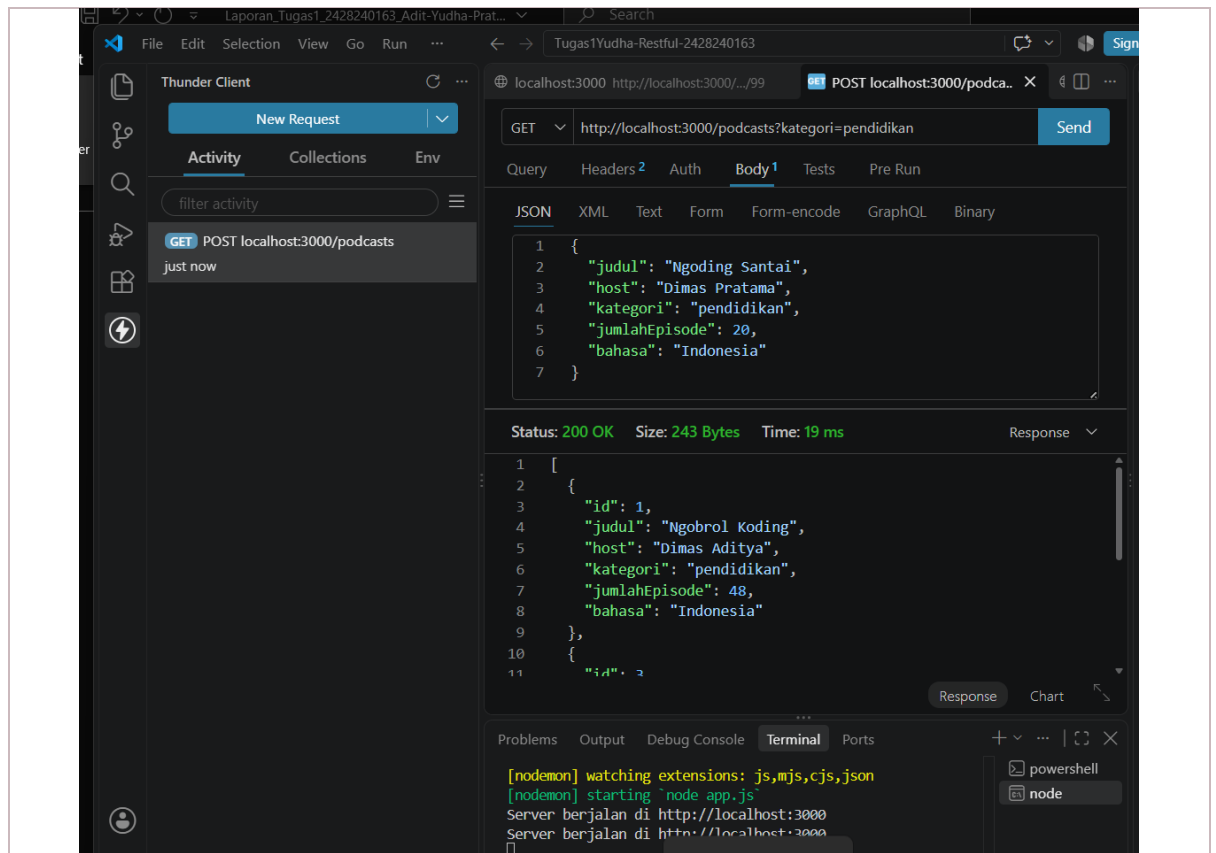
Gambar 2. GET /podcasts/1 (ambil satu data) (status diharapkan 200)

Pembahasan: Nilai id diambil dari route parameter `req.params.id`, diubah menjadi angka dengan `Number()`, lalu dicari dengan `find()`. Karena podcast dengan id 1 ("Ngobrol Koding") tersedia, server membalas status 200 dengan satu objek podcast.

Pengujian 3 — GET /podcasts/99 (id tidak ditemukan)

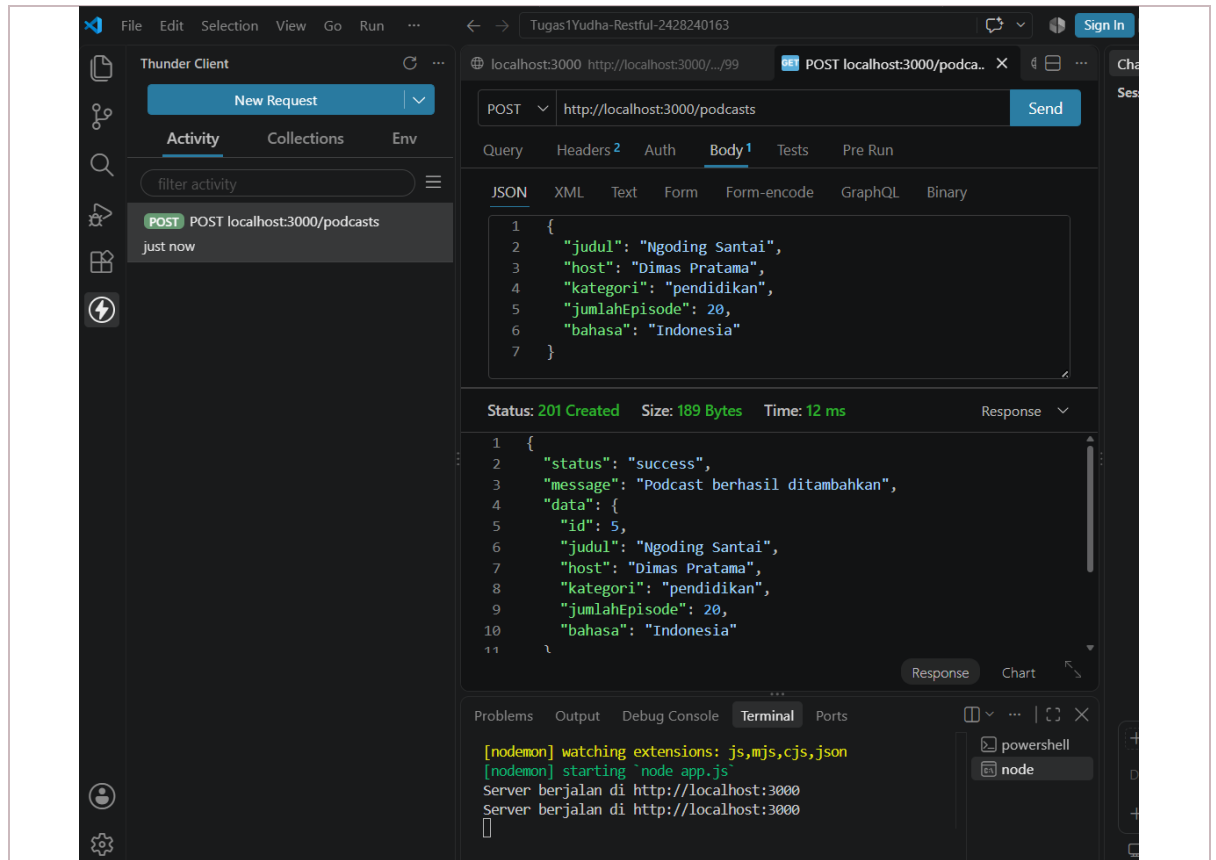
Gambar 3. GET /podcasts/99 (id tidak ditemukan) (status diharapkan 404)

Pembahasan: Podcast dengan id 99 tidak ada di array, sehingga server membalas status 404 (Not Found) dengan struktur `{ status: "error", message, data: null }`. Status 404 dipilih karena resource yang diminta memang tidak ada, bukan karena format request yang salah.

Pengujian 4 — GET /podcasts?kategori=pendidikan (filter query string)

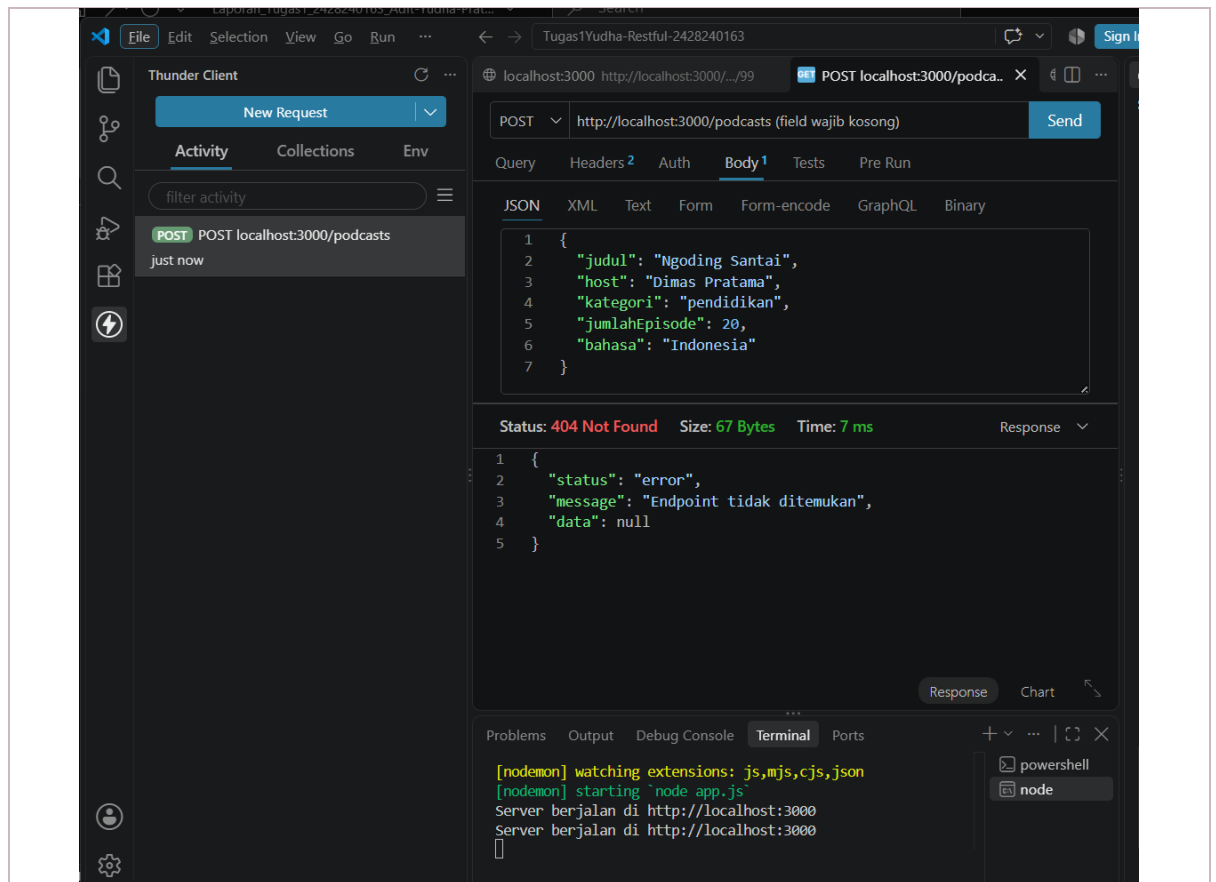
Gambar 4. GET /podcasts?kategori=pendidikan (filter query string) (status diharapkan 200)

Pembahasan: Filter memakai query string (`req.query.kategori`), bukan route parameter, sesuai ketentuan tugas. Pencocokan memakai `toLowerCase()` sehingga tidak peka huruf besar-kecil. Hasilnya berupa array dan tetap berstatus 200 meskipun kosong (`[]`), karena request valid tetapi tidak ada data yang cocok.

Pengujian 5 — POST /podcasts (body lengkap)

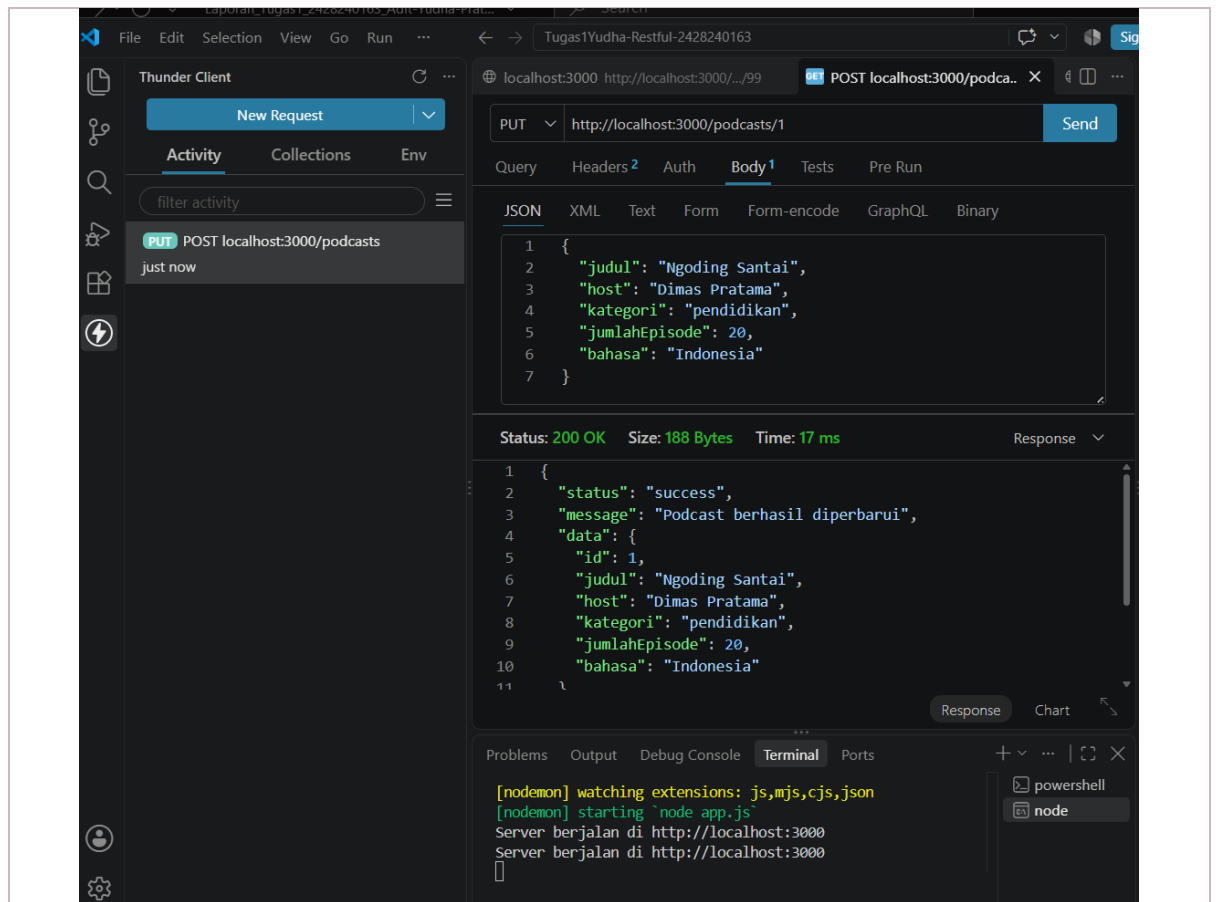
Gambar 5. POST /podcasts (body lengkap) (status diharapkan 201)

Pembahasan: POST dipakai untuk membuat resource baru. Server memvalidasi field wajib (judul, host, dan kategori), membuat id otomatis dengan `nextId++`, lalu menyimpan data ke array. Status 201 (Created) dipilih karena resource baru berhasil dibuat; body berisi status: "success", message, dan data yang baru dibuat.

Pengujian 6 — POST /podcasts (field wajib kosong)

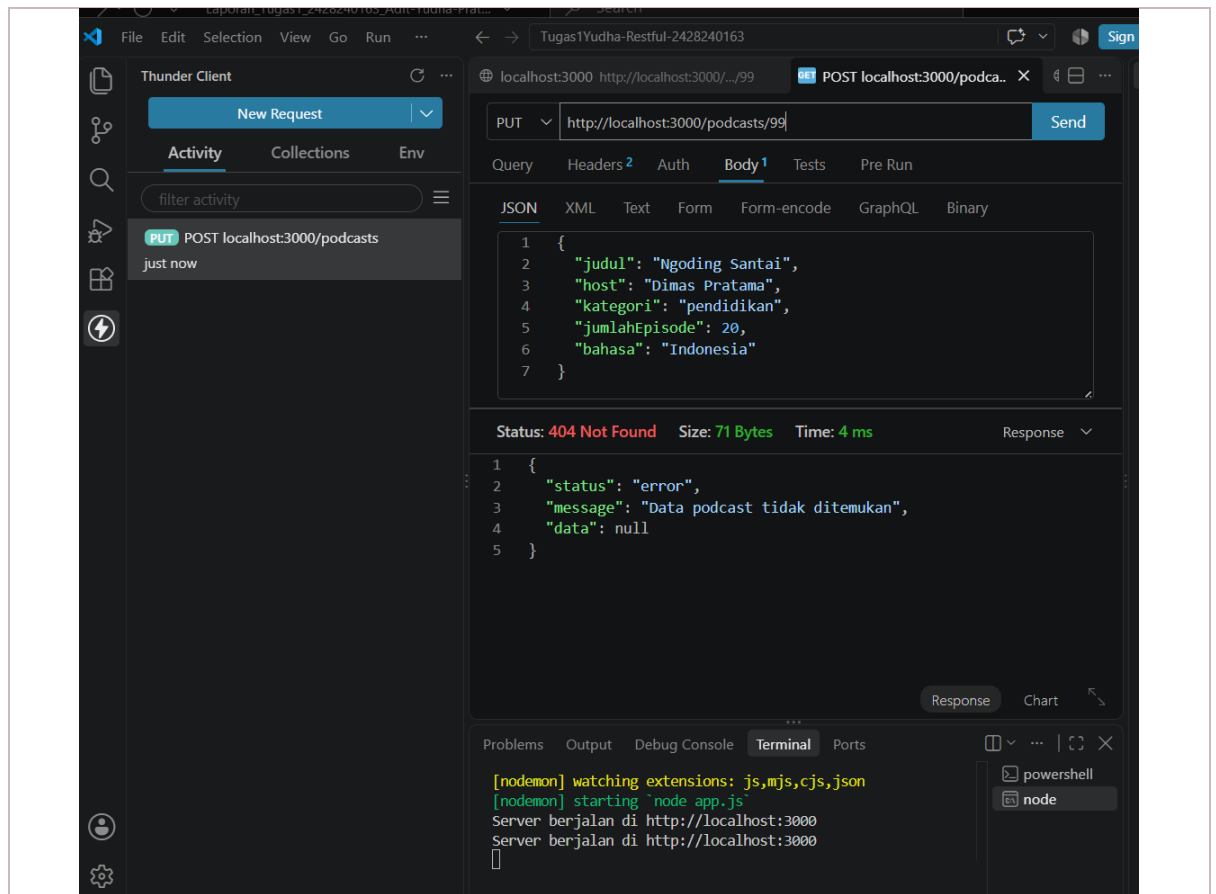
Gambar 6. POST /podcasts (field wajib kosong) (status diharapkan 400)

Pembahasan: Field wajib diperiksa dengan fungsi `isEmpty()` yang juga menolak string berisi spasi saja. Karena field wajib kosong, server membalas status 400 (Bad Request) dengan data: `null`. Data tidak ditambahkan dan `nextId` tidak bertambah karena proses berhenti sebelum data dibuat.

Pengujian 7 — PUT /podcasts/1 (body lengkap)

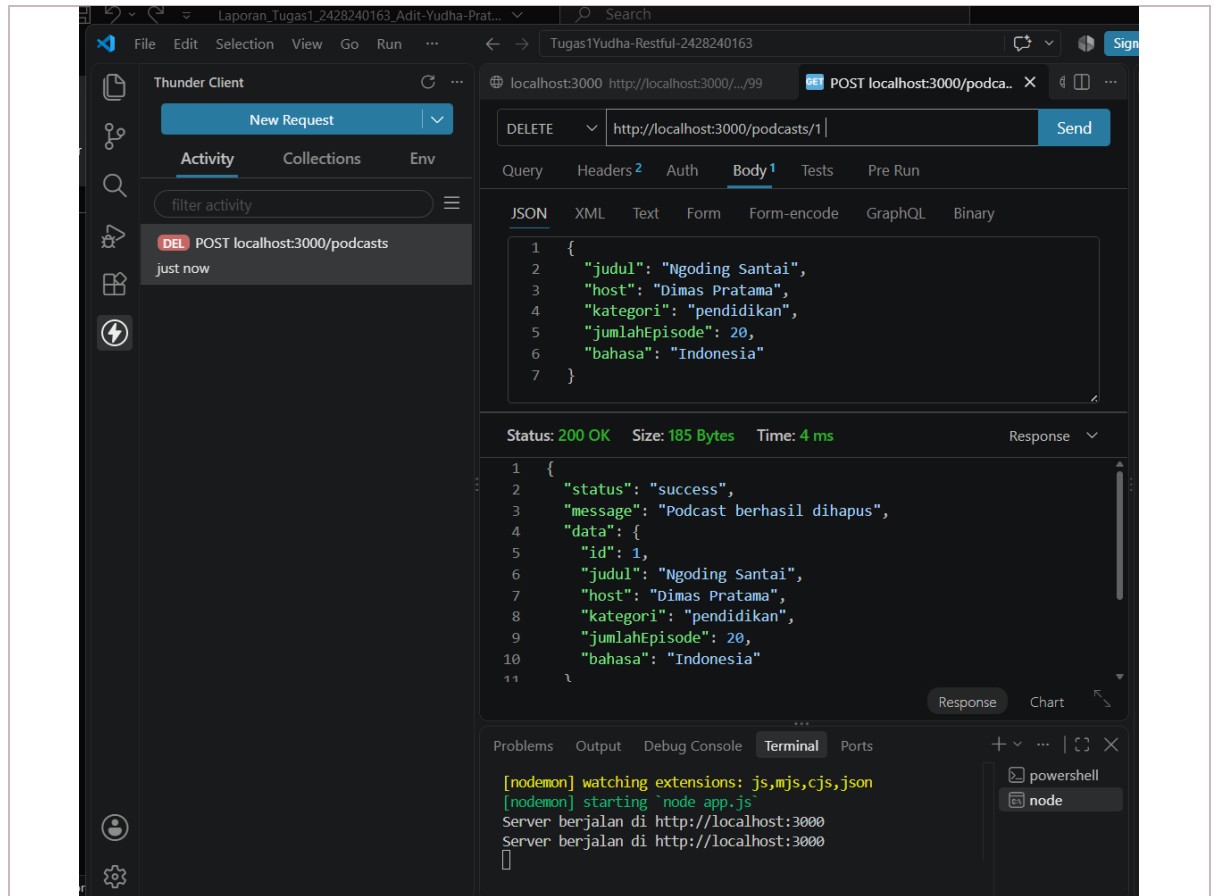
Gambar 7. PUT /podcasts/1 (body lengkap) (status diharapkan 200)

Pembahasan: PUT mengganti seluruh data, sehingga semua field wajib harus dikirim ulang. Objek lama diganti utuh dengan objek baru yang tetap memakai id yang sama. Status 200 (OK) dipilih karena data yang sudah ada berhasil diperbarui, bukan 201 karena tidak ada resource baru yang dibuat.

Pengujian 8 — PUT /podcasts/99 (id tidak ditemukan)

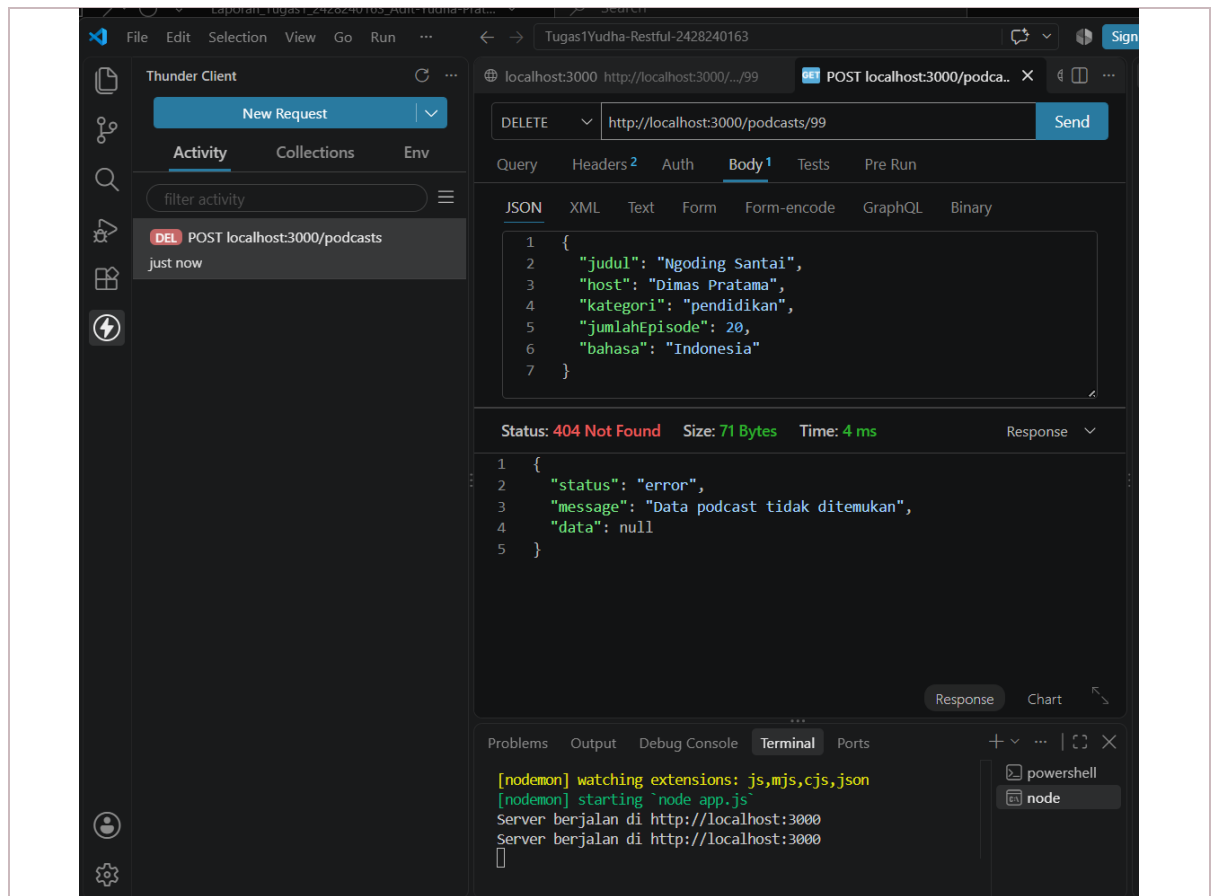
Gambar 8. PUT /podcasts/99 (id tidak ditemukan) (status diharapkan 404)

Pembahasan: Server memeriksa keberadaan data dengan `findIndex()` terlebih dahulu. Karena hasilnya `-1`, server membalas status `404 (Not Found)` tanpa memproses body request.

Pengujian 9 — DELETE /podcasts/1 (hapus data)

Gambar 9. DELETE /podcasts/1 (hapus data) (status diharapkan 200)

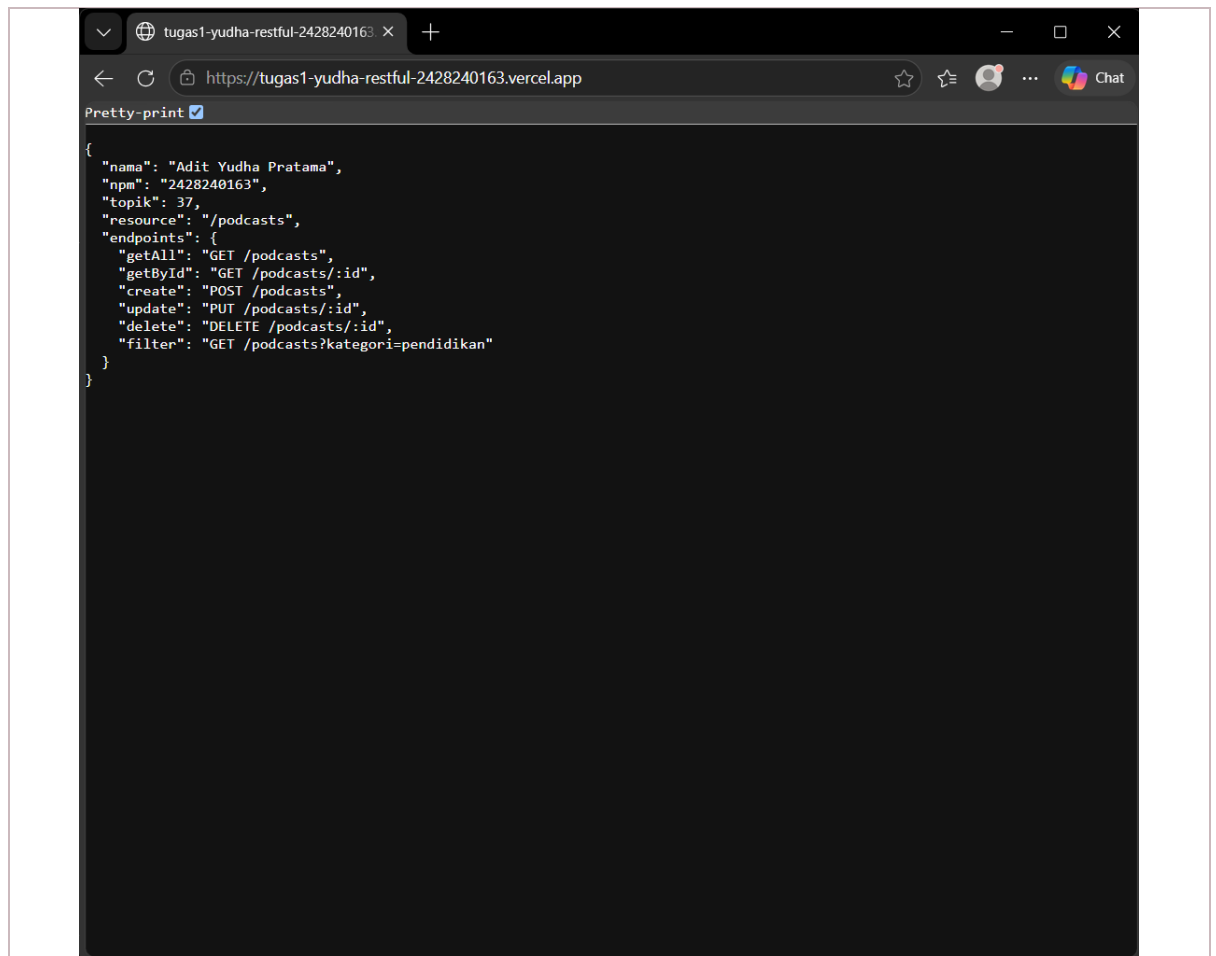
Pembahasan: DELETE dipakai untuk menghapus resource; data dihapus dari array dengan `splice()`. Status 200 (OK) dipilih karena penghapusan berhasil. Sesuai Soal 4, message menyebut id yang dihapus dan data bernilai null.

Pengujian 10 — DELETE /podcasts/99 (id tidak ditemukan)

Gambar 10. DELETE /podcasts/99 (id tidak ditemukan) (status diharapkan 404)

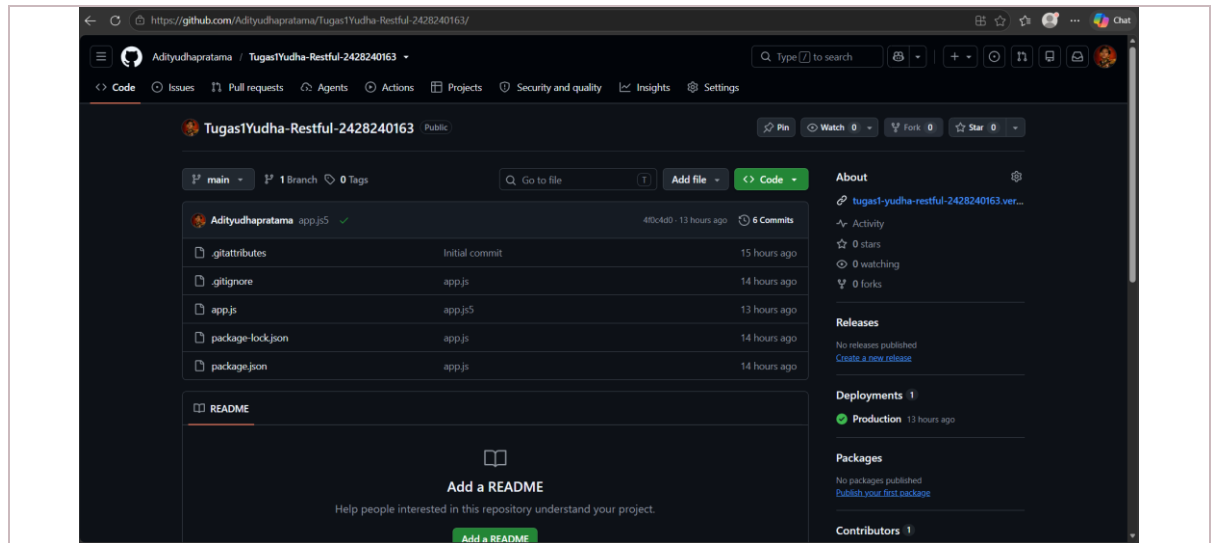
Pembahasan: Podcast dengan id 99 tidak ditemukan sehingga tidak ada yang dapat dihapus. Server membalas status 404 (Not Found) dengan data: null dan array podcasts tidak berubah.

Pengujian pada URL Vercel



Gambar 11. Pengujian endpoint pada URL Vercel: [tugas1-yudha-restful-2428240163.vercel.app].

Struktur endpoint yang ditampilkan di Vercel **sama dengan endpoint yang digunakan pada localhost**, sehingga secara struktur API sudah sesuai dengan implementasi sebelumnya. **Riwayat Commit GitHub**



Gambar 12. Riwayat commit repository tugas1-restful-2428240163

Pembahasan Umum

Pemilihan method mengikuti semantik HTTP (Fielding et al., 2022): GET bersifat hanya-baca, POST membuat resource baru dengan id yang dibuat server, PUT mengganti seluruh data sehingga seluruh field wajib harus dikirim, dan DELETE menghapus data berdasarkan id. Status code dipilih sesuai hasilnya: 200 untuk operasi baca, ubah, dan hapus yang berhasil; 201 untuk data baru yang berhasil dibuat; 400 untuk validasi yang gagal; serta 404 untuk data atau endpoint yang tidak ditemukan.

Selain itu, middleware catch-all di akhir file membalas endpoint yang tidak terdaftar dengan status 404 dan pesan "Endpoint tidak ditemukan", sedangkan error handler membalas JSON yang tidak valid dengan status 400. Dengan demikian seluruh response, termasuk response error, tetap berformat JSON dan tidak ada `res.send()` teks/HTML maupun `res.render()`.

VI. Kendala dan Solusi

No.	Kendala	Solusi
1	Setelah deploy, endpoint tidak dapat diakses karena <code>`app.listen()`</code> tidak cocok untuk lingkungan serverless Vercel.	<code>`app.listen()`</code> hanya dijalankan bila <code>`NODE_ENV`</code> bukan production, dan aplikasi diekspor dengan <code>`module.exports = app`</code> .
2	Data podcast kembali ke data awal setelah beberapa saat di Vercel.	Data disimpan pada array di memori dan Vercel bersifat serverless, sehingga hal ini wajar. Pengujian cukup memastikan setiap request mendapat response yang benar.
3		

VII. Kesimpulan

Praktikum ini berhasil membangun RESTful API murni dengan Express.js untuk resource podcast (Topik 37) yang terdiri dari satu file app.js, data dalam array di memori, dan 6 endpoint (GET semua data, GET satu data, POST, PUT, DELETE, serta filter dengan query string ?kategori=). Hal ini sesuai dengan tujuan 1 dan 2.

Method GET, POST, PUT, dan DELETE digunakan sesuai fungsinya, disertai validasi field wajib dan status code 200, 201, 400, serta 404 (tujuan 3). Seluruh response berformat JSON murni dengan struktur { status, message, data } pada POST, PUT, DELETE, dan error (tujuan 4). Berdasarkan pengujian, seluruh skenario memberikan status sesuai yang diharapkan — sesuaikan kalimat ini bila ada skenario yang tidak sesuai.

Project telah dipublikasikan ke GitHub dan di-deploy ke Vercel sehingga dapat diuji pada URL publik (tujuan 5). Karena data hanya disimpan di memori, data akan kembali ke kondisi awal saat aplikasi dijalankan ulang; untuk penyimpanan permanen diperlukan database.

Daftar Pustaka

- Bray, T. (Ed.). (2017). *The JavaScript Object Notation (JSON) data interchange format* (RFC 8259). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc8259>
- Express. (n.d.). *Express – Node.js web application framework*. <https://expressjs.com/>
- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine]. <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022). *HTTP semantics* (RFC 9110). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc9110>
- Vercel. (n.d.). *Vercel documentation*. <https://vercel.com/docs>

Lampiran

Tautan Repository GitHub:

[ISI: <https://github.com/USERNAME-ANDA/tugas1-restful-2428240163>]

Tautan Deployment (jika ada):

[ISI: URL Vercel, contoh <https://tugas1-restful-2428240163.vercel.app>]